DIGITAL EGYPT PIONEERS INITIATIVE (DEPI)
DATA ENGINEERING TRACK - PHASE 5
GRADUATION PROJECT DOCUMENTATION (PHASE 1)
==========

Project Title: E-Commerce Data Integration & Analytics Pipeline

---
TEAM MEMBERS & RESPONSIBILITIES

• Team Leader / Orchestration Engineer:
Managing overall project workflow, milestones tracking, GitHub repository structure, and Apache Airflow DAG scheduling.
• Data Engineer (Ingestion Specialist):
Developing REST API extraction scripts, parsing local CSV files, and managing the raw data storage layer.
• Data Engineer (Transformation & Quality Specialist):
Handling Pandas-based data cleaning, schema validation, null value management, and error log routing.
• Database & Modeling Engineer:
Designing the relational Star Schema (Fact and Dimension tables), writing SQL DDL scripts, and optimizing analytical queries.

---
1. PROJECT PLANNING & MANAGEMENT

1.1 Idea, Objectives, and Scope
• Project Idea: Our team is developing an automated end-to-end data engineering pipeline designed for an e-commerce enterprise. The system consolidates fractured datasets from external REST APIs and local CSV files, cleans and validates the records, structures them into a dimensional data mart, and automates execution cycles via workflow orchestration.
• Core Objectives:
1. Extract raw transactional, product, and customer records reliably from diverse sources.
2. Implement robust data cleaning, type casting, and validation routines using Python and Pandas.
3. Design a scalable Star Schema data mart to support business intelligence reporting and analytical queries.
4. Automate incremental pipeline execution and monitor task states using Apache Airflow.
5. Deliver a clean, production-ready, version-controlled analytical repository.
• Scope: The pipeline handles batch and incremental loads for sales orders, customer profiles, and product catalogs. Real-time stream processing frameworks (such as Apache Kafka) are intentionally excluded from the initial scope to maintain focus on core batch engineering best practices.

1.2 Timeline & Implementation Plan
• Week 1 (Current Phase): Requirements gathering, team role alignment, repository initialization, and Phase 1 documentation.
• Week 2: Development of core ingestion modules (API fetchers & CSV parsers) and staging layer setup.
• Week 3: Implementation of Pandas transformation scripts, data quality assertion checks, and relational Star Schema creation in the database.
• Week 4: Integration of Apache Airflow DAGs, task dependencies scheduling, and error-handling mechanisms.
• Week 5: System integration testing, repository clean-up, documentation finalization, and presentation preparation.

1.3 Risk Management & Mitigation
• API Rate Limits & Timeouts: Mitigated by writing custom retry logic with exponential backoff and implementing local raw data caching.
• Schema Inconsistencies & Bad Data: Handled through pre-loading assertions that filter, isolate, and log malformed rows instead of breaking the entire pipeline run.
• Orchestration Failures: Managed by configuring explicit task sequencing, retries, and failure alerts within Airflow.

1.4 Key Performance Indicators (KPIs)
• Data Integrity: 100% removal of duplicate orders and invalid primary/foreign keys in the curated data mart.
• Pipeline Reliability: Maintaining a 99%+ successful execution rate across automated scheduler cycles.

---

2. LITERATURE REVIEW

2.1 Comparative Analysis
• Manual Cron Scripts vs. Our Approach: Traditional shell scripts lack dependency tracking, state management, and clear monitoring. Our approach integrates Apache Airflow to handle task retry logic, dynamic scheduling, and operational visibility.
• SaaS ETL Tools vs. Custom Pipeline: Managed integration tools offer quick setup but introduce recurring licensing costs and vendor lock-in. Building a custom Python and Airflow solution gives our team complete control over custom business logic, data validation rules, and zero-cost scalability.

2.2 Value Proposition
By bridging chaotic raw e-commerce inputs with a structured dimensional warehouse, this project directly solves common enterprise data quality issues, ensuring analysts receive clean, high-integrity data ready for immediate business intelligence consumption.

---

3. REQUIREMENTS GATHERING

3.1 Stakeholder Analysis

• Data Analysts & BI Engineers: Require clean, structured, and query-optimized dimensional tables to build performance dashboards without data anomalies.
• Business Operations Managers: Require timely, accurate daily updates regarding sales metrics, customer purchasing patterns, and inventory status.
• Data Engineering Team: Requires modular code structure, descriptive execution logs, and robust pipeline observability.

3.2 Functional Requirements

1. Ingestion Layer: The system must connect to external REST API endpoints and parse local CSV/JSON files containing e-commerce transaction logs.
2. Data Cleaning: The pipeline must validate data types, inspect for null constraints, strip duplicates, and direct error records to an isolation table.
3. Dimensional Modeling: Transformed datasets must be systematically loaded into a Star Schema consisting of Fact_Sales, Dim_Customer, Dim_Product, and Dim_Date tables.
4. Incremental Loading: The pipeline must process only newly updated or inserted records since the previous successful execution window.
5. Orchestration: Apache Airflow must sequence and automate the extract, transform, load, and validate operations cleanly.

3.3 Non-Functional Requirements

• Performance: Daily batch processing pipelines must complete their execution windows efficiently under standard hardware constraints.
• Maintainability: Source code must adhere to clean modular Python conventions (PEP 8), be tracked under Git, and be accompanied by comprehensive technical documentation.